

React Dev Camp Session 3

From Components to Data



Quick Recap

Last session:

- TypeScript basics
- `useState`
- Rendering & re-rendering
- Todo app with state



Session Goals

Today we'll look at:

- React structure
- `useEffect`
- `fetch`
- API data
- Passing state between components



React Apps Grow Quickly

It starts simple 📌

```
function App() {  
  return <h1>Hello world</h1>;  
}
```



Then This Happens

- State
- Fetching
- Lists
- Forms
- Event handlers
- Components
- Styling

Everything ends up in one file.



Why Structure Matters

Good structure helps with:

- Readability
- Reusability
- Team collaboration
- Maintainability



Example Structure

```
src/  
  components/  
    PokemonList.tsx  
    PartyList.tsx  
  
  types/  
    pokemon.ts  
  
App.tsx
```



Introducing useEffect

`useEffect` lets us run code:

- After rendering
- On component load
- When values change



useEffect structure

```
function useEffect(  
  effect: () => void | (() => void),  
  deps?: readonly unknown[]  
): void;
```

- **The Effect Callback:** Must return either void or a cleanup function that returns void.
- **The Return Value:** You cannot return a Promise. This means the callback function cannot be marked as async.
- **Dependencies:** An optional read-only array containing any data type (unknown[]).



Basic useEffect

```
useEffect(() => {  
  console.log("Component loaded");  
}, []);
```

- Only run on initial render of the page
- Logs `Component loaded` only when the page first renders
- Does not run on each re-render of the component



Dependency Array

```
useEffect(() => {  
  console.log("Runs when count changes");  
}, [count]);
```

- Empty array → run once
- Dependencies → run when values change
- Does not run until the value of `count` changes



Data Fetching

Frontend applications often need:

- API calls
- Server data
- Async behaviour



fetch Example

`fetch` is the browser's built-in HTTP client.

```
const response = await fetch("https://pokeapi.co/api/v2/pokemon");  
const data = await response.json();
```

- Make a GET request to the PokéApi
- Store the response in `response`
- Parses that response from json into an object
- Store that result in `data`



fetch Returns a Promise

A Promise means:

“The result will come later”

API calls take time.



Using .then()

```
fetch("https://pokeapi.co/api/v2/pokemon")  
  .then((response) => response.json())  
  .then((data) => console.log(data));
```



async/await Example

```
async function fetchPokemon() {  
  const response = await fetch(  
    "https://pokeapi.co/api/v2/pokemon"  
  );  
  
  const data = await response.json();  
  
  console.log(data);  
}
```



fetch + useEffect

```
useEffect(() => {  
  async function fetchPokemon() {  
    const response = await fetch(  
      "https://pokeapi.co/api/v2/pokemon"  
    );  
  
    const data = await response.json();  
  
    setPokemon(data.results);  
  }  
  
  fetchPokemon();  
}, []);
```




React Flow

Component loads



useEffect runs



fetch gets data



State updates



React re-renders



UI updates



Important Rule

Rendering should describe UI.

Side effects should happen in `useEffect` .

Avoid This

```
const response = await fetch( ... );
```

Inside the component body.

💥 Why?

Components re-render often.

This could cause repeated API calls.



Practical

We'll build:

- A Pokédex list
- A Pokémon party builder
- Max 6 Pokémon in the party



Features

- Fetch Pokémon from API
- Render list
- Add Pokémon to party
- Prevent more than 6 Pokémon



What's Next

Later sessions:

- Services and where to put API calls
- DRY (Don't Repeat Yourself)
- Custom hooks
- Pagination
- More data in our Pokédex



Wrap-up

- `useEffect` handles side effects
- `fetch` lets us retrieve data
- Structure becomes important as apps grow